
Planning Intensif – 1 Semaine

Systeme de Monitoring des Infrastructures Reseaux et Serveurs de l'EPT – Version complete

Etudiants : Khadija DIENG & Boubacar SIDIBE
Periode : Vendredi 28 aout – Jeudi 3 septembre 2026 (7 jours)
Charge : Temps plein, 6 a 8h/jour, les deux etudiants en parallele
Objectif : Couvrir F1 a F8 du cahier des charges, systeme deploye et de-
montrable

Avertissement realiste – a lire avant de commencer

En 7 jours a deux, meme a temps plein, vous pouvez livrer un systeme **complet et fonctionnel de bout en bout**. Vous ne pouvez pas atteindre le meme niveau de profondeur qu'un planning de 24 semaines sur : couverture exhaustive des diagrammes UML, tests automatises a haute couverture, audit de securite pousse, tests de charge reels, documentation tres detaillee. La section 3 liste precisement ce qui est volontairement allége cette semaine, pour que ce soit un choix assume et non une surprise en soutenance.

Contents

1	Principes d'organisation de la semaine	2
2	Planning jour par jour	3
3	Ce qui est volontairement allège cette semaine (à assumer en soutenance)	7

1 Principes d'organisation de la semaine

- **Deux pistes en parallele, en permanence.** Etudiant 1 (Backend / Donnees / Reseau) et Etudiant 2 (Collecte / Frontend / Livrables). Adaptez les noms a vos affinites reelles, mais ne travaillez jamais tous les deux sur la meme brique en meme temps – vous perdez des heures precieuses.
- **Un point de synchronisation chaque soir**, 30 minutes maximum : ce qui marche, ce qui bloque, integration testee.
- **On code directement sur develop** cette semaine (pas le temps de multiplier les pull requests), mais on commit toutes les 1-2h avec des messages clairs, pour pouvoir revenir en arriere en cas de casse.
- **Scope fige des le depart.** On ne rajoute rien qui n'est pas dans la liste du jour. Toute idee bonus va dans un fichier `IDEES_FUTURES.md`, pas dans le code.
- **Chaque jour se termine sur un critere de validation concret** (section grisee "critere de fin de journee"). S'il n'est pas atteint, la priorite du lendemain matin est de le combler avant d'avancer.
- **Les donnees de demonstration se preparent au fur et a mesure**, pas le dernier jour : gardez toujours au moins un serveur/VM de test et un equipement reseau (reel ou simule) qui tournent en continu depuis le Jour 2.

2 Planning jour par jour

JOUR 1 – Vendredi 28 aout – Fondations communes puis demarrage parallele

MATIN (ensemble, 3h) – fondations non negociables

1. Depot Git (`monitoring-ept`), structure `/backend /frontend /agent-collecte /collecteur-snmp /infra /docs`.
2. Modele de donnees minimal mais complet : `users, roles, equipments, metrics, alerts, reports` – esquisse rapide (papier ou Draw.io, 30 min max, pas de formalisme UML exhaustif).
3. `docker-compose.yml` avec service PostgreSQL + volume ; base creee.
4. Repartition definitive des taches restantes entre les deux etudiants (voir pistes ci-dessous).

Etudiant 1 – Backend / Donnees / Reseau (apres-midi)

- Init projet Spring Boot (Web, Data JPA, Security, PostgreSQL, Flyway).
- Entites JPA + migration Flyway du schema du matin.
- Endpoint `POST /auth/login` + generation JWT + 3 roles (Administrateur, Technicien, Observateur).

Etudiant 2 – Collecte / Frontend (apres-midi)

- Init agent Python (`venv, psutil, requests`) : collecte CPU/RAM/disque en boucle, envoi HTTP vers une URL mockee.
- Init projet React (Vite) : structure `/pages /components /services /context`, page de connexion (design simple, sans backend reel pour l'instant).

SYNC DU SOIR (30 min, obligatoire)

Demo croisee : login JWT teste via Postman ; agent Python affiche ses mesures collectees dans la console.

CRITERE DE FIN DE JOURNEE (ne pas dormir avant que ce soit vert)

Le depot compile/demarre des deux cotes ; `docker compose up` lance Postgres sans erreur ; login JWT fonctionnel en Postman.

JOUR 2 – Samedi 29 aout – CRUD equipements et debut de la collecte reelle

Etudiant 1

- CRUD complet `/api/equipments` (F2) : creation, modification, suppression, consultation, recherche par nom/IP/type.
- Endpoint `POST /api/metrics/server` pour reception des metriques agent, avec cle API dediee (pas de JWT utilisateur pour les agents).
- Test avec Postman : creer 2-3 equipements de test.

Etudiant 2

- Connecter l'agent Python au vrai endpoint backend (cle API), ajouter gestion d'erreur reseau basique (retry).
- Installer/lancer l'agent sur une VM Linux de test (VirtualBox/Vagrant ou VM CRI) en continu – il doit tourner jusqu'à la fin de la semaine.
- Demarrer le collecteur SNMP : `pysnmp`, requetes de base sur un equipement de test ou simulateur `snmpsim` (disponibilite via `sysUpTime`, etat interface `ifOperStatus`).

SYNC DU SOIR (30 min, obligatoire)

Verifier ensemble que les metriques de l'agent apparaissent bien en base PostgreSQL (requete SQL directe).

CRITERE DE FIN DE JOURNEE (ne pas dormir avant que ce soit vert)

Au moins un serveur de test envoie des metriques reelles stockees en base toutes les 30-60s ; CRUD equipements 100% fonctionnel en Postman.

JOUR 3 – Dimanche 30 aout – Reseau, WebSocket et premier ecran connecte**Etudiant 1**

- Endpoint `POST /api/metrics/network` (F4) pour reception des donnees SNMP/ICMP.
- Configuration WebSocket STOMP (`/ws`), topics `/topic/metrics/{id}` et `/topic/alerts` ; publication automatique a chaque insertion.
- Endpoint de consultation des metriques avec filtres (equipement, plage de temps) pour alimenter les graphiques.

Etudiant 2

- Connecter le collecteur SNMP au vrai endpoint backend ; le laisser tourner en continu des maintenant.
- Frontend : page "Vue d'ensemble" (etat global) + page "Equipements" (liste + recherche) branchees sur l'API reelle.
- Integration client WebSocket (`@stomp/stompjs`) pour mise a jour live d'un premier comp-teur/carte de statut.

SYNC DU SOIR (30 min, obligatoire)

Test integration complet : modifier une metrique cote backend (ou attendre la collecte reelle) et verifier l'affichage temps reel cote frontend en moins de 3 secondes.

CRITERE DE FIN DE JOURNEE (ne pas dormir avant que ce soit vert)

Chaine complete Agent/SNMP → Backend → WebSocket → Frontend demonstrable sans intervention manuelle.

JOUR 4 – Lundi 31 aout – Alertes et dashboard visuel**Etudiant 1**

- Moteur de regles d'alerte simple (F6) : seuils CPU/RAM/disque, equipement injoignable – evaluation a chaque insertion de metrique, creation d'**Alert** avec anti-doublon.
- Publication des alertes sur `/topic/alerts` ; endpoint d'acquittement `PATCH /api/alerts/{id}/ack`.
- Envoi d'e-mail basique pour les alertes critiques (SMTP de test type Mailtrap) (F7 – volet e-mail uniquement).

Etudiant 2

- Fiche detail d'un equipement avec graphiques d'evolution (Recharts/Chart.js) sur les dernieres 24h.
- Page "Alertes" : liste temps reel via WebSocket + bouton d'acquittement.
- Affichage conditionnel par role (un Observateur ne voit pas les boutons d'action).

SYNC DU SOIR (30 min, obligatoire)

Simuler une surcharge CPU sur la VM de test (`stress-ng -cpu 4 -timeout 120s`) et verifier : alerte generee, visible en moins de 3s dans l'UI, e-mail reçu.

CRITERE DE FIN DE JOURNEE (ne pas dormir avant que ce soit vert)

Alerte declenchee, affichee, acquittable et notifiee par e-mail sur un incident reel simule.

JOUR 5 – Mardi 1er septembre – Rapports et durcissement securite**Etudiant 1**

- Endpoint de generation de rapport a la demande (F8) : agregation SQL (moyennes/min/-max/comptage alertes) sur une periode donnee, export PDF simple (OpenPDF/iText) ou CSV si le temps manque.
- Durcissement securite minimal : verification que tous les mots de passe sont bien en BCrypt, limitation basique des tentatives de login, CORS restreint au domaine frontend, variables sensibles sorties du code (`.env`).

Etudiant 2

- Page "Rapports" frontend : generation a la demande + telechargement.
- Passage rapide de polish UI (coherence visuelle, responsive basique desktop/tablette) sur toutes les pages existantes.
- Preparer et enrichir le jeu de donnees de demonstration (equipements nommes clairement, quelques jours d'historique si possible en accelerant artificiellement la collecte pour le jeu de demo).

SYNC DU SOIR (30 min, obligatoire)

Verifier que le rapport genere contient des chiffres coherents avec un controle manuel sur un echantillon ; verifier le controle d'accès par role sur 2-3 actions sensibles.

CRITERE DE FIN DE JOURNEE (ne pas dormir avant que ce soit vert)

Rapport telechargeable avec donnees coherentes ; un Observateur ne peut ni modifier un equipement ni acquitter une alerte (verifie manuellement).

JOUR 6 – Mercredi 2 septembre – Deploiement et tests globaux (journee commune)**MATIN (ensemble, 4h) – deploiement**

1. `docker-compose.prod.yml` : backend, frontend (build statique), PostgreSQL, avec `restart: unless-stopped`.
2. Nginx en reverse proxy : route `/api` vers le backend, sert le frontend, en-tetes WebSocket (`Upgrade/Connection`) correctement configures.
3. Deploiement sur l'environnement cible (serveur CRI si disponible, sinon VM de test qui simule la production) ; agent et collecteur SNMP pointes vers les vraies adresses si possible.

APRES-MIDI (ensemble, 4h) – tests bout-en-bout

1. Checklist de test rapide sur F1 a F8, un scenario par fonctionnalite, note PASS/FAIL.
2. Correction des bugs bloquants uniquement (report des bugs mineurs dans `IDEES_FUTURES.md` ou un backlog "connu, non bloquant").
3. Verification que la plateforme deployee tient sans intervention pendant plusieurs heures (la laisser tourner pendant la soiree/la nuit).

CRITERE DE FIN DE JOURNEE (ne pas dormir avant que ce soit vert)

Plateforme deployee accessible par URL/IP fixe ; checklist F1-F8 executee avec au maximum des anomalies mineures documentees, aucune bloquante.

JOUR 7 – Jeudi 3 septembre – Documentation, rapport et repetition de soutenance**Etudiant 1 (matin)**

- Documentation technique condensee : architecture finale, schema de la base, liste des endpoints principaux (export Swagger si le temps le permet), procedure d'installation.
- Verification finale que la plateforme deployee tourne toujours (elle a tourne toute la nuit).

Etudiant 2 (matin)

- Guide utilisateur condense (captures d'ecran) par role : Administrateur, Technicien, Observateur.
- Captures d'ecran / courte video de secours du scenario de demo, au cas ou le reseau flanche le jour J.

APRES-MIDI ET SOIR (ensemble)

1. Rediger/finaliser le rapport de projet (reutiliser la documentation du matin, structure academique standard).
2. Preparer les slides de soutenance : probleme, architecture, demonstration, resultats, limites assumees de la semaine (section 3), perspectives (application mobile, Telegram, SMS – deja identifiees comme evolutions futures dans le cahier des charges).
3. Repetition complete du scenario de demonstration, au moins deux fois a l'identique.
4. Repetition orale chronometree.

CRITERE DE FIN DE JOURNEE (ne pas dormir avant que ce soit vert)

Rapport et slides finalises ; demo repete deux fois sans blocage ; plan B (captures/video) pret en cas d'incident technique.

3 Ce qui est volontairement allège cette semaine (a assumer en soutenance)

Domaine	Ce qui est fait cette semaine	Ce qui est allège vs. version ideale
Conception UML	Modele de donnees + schema d'architecture	Pas de diagrammes de cas d'utilisation/sequence exhaustifs et formalises
Reseau SNMP	Disponibilite, etat interface, quelques OID cles	Couverture MIB limitee, pas tous les equipements du CRI testes
Tests automatises	Tests manuels cibles sur les scenarios critiques	Pas de suite automatisee a forte couverture (JUnit/Jest complets)
Securite	JWT, BCrypt, roles, CORS, limitation login basique	Pas d'audit de securite formel (OWASP), pas de tests de penetration
Performance	Verification empirique du temps de reponse	Pas de test de charge outille (JMeter/k6) avec rapport chiffre
Notifications	E-mail et notifications web	Application mobile, Telegram, SMS non implementes (deja notes comme evolutions futures)
Documentation	Doc technique et utilisateur condensees	Pas de documentation exhaustive de maintenance long terme
Rapports	Generation a la demande	Planification automatique (journalier/hebdo/mensuel) probablement repousse si le temps manque

Conseil pour la soutenance

Presentez ce tableau vous-memes, avant qu'on ne vous le demande. Un jury valorise davantage une equipe qui a fait des choix de priorisation assumes et documentes qu'une equipe qui pretend avoir tout fait parfaitement en une semaine.